

Lesson5

面向对象的三大特称之二：继承 (inheritance)



主要内容

- 继承的必要性
- 理解继承
 - 基类与派生类的关系
 - 单继承、多继承
 - 继承的层次结构
 - 子类继承了什么？
- Java继承（extends）
 - 子类构造方法
 - 保护成员的使用
 - 向上映射
 - 隐藏、覆盖
- 继承的优缺点、组合与继承
- 继承的案例

继承的必要性

- 假设现在定义了一个小鹿类，具有吃饭和睡觉两个方法。
- 嗯，下一步：直接实例化这个Deer对象，就可以得到吃草的并且是躺着睡觉的小鹿。

```
class Deer {  
    public void eat(){  
        System.out.println(x: "我在吃草");  
    }  
  
    public void sleep(){  
        System.out.println(x: "我在躺着睡觉");  
    }  
}
```

思考：

- 问题1. 想new出一个狮子，但是狮子是吃肉且躺着睡觉的，怎么办？
- 问题2. 想new出一个吃肉，躺着睡觉，并且还可以飞的动物，怎么办？

Answer 1

- 第一个问题：要new一个狮子，还是吃肉和躺着睡觉的。但是我们没有狮子类啊，怎么办？简单，新建一个Lion类，定义一下就好啦：

```
class Lion {  
    public void eat(){  
        System.out.println(x: "我在吃肉");  
    }  
  
    public void sleep(){  
        System.out.println(x: "我在躺着睡觉");  
    }  
}
```

Answer 2

- 第二个问题：要new一个吃肉，躺着睡觉，并且还可以飞的老鹰，怎么办？也简单，我们再new 一个Eagle类出来就好啦：

```
class Eagle {  
    public void eat(){  
        System.out.println(x: "我在吃肉");  
    }  
  
    public void sleep(){  
        System.out.println(x: "我在躺着睡觉");  
    }  
  
    public void fly(){  
        System.out.println(x: "I am flying!");  
    }  
}
```

解决方案的利弊

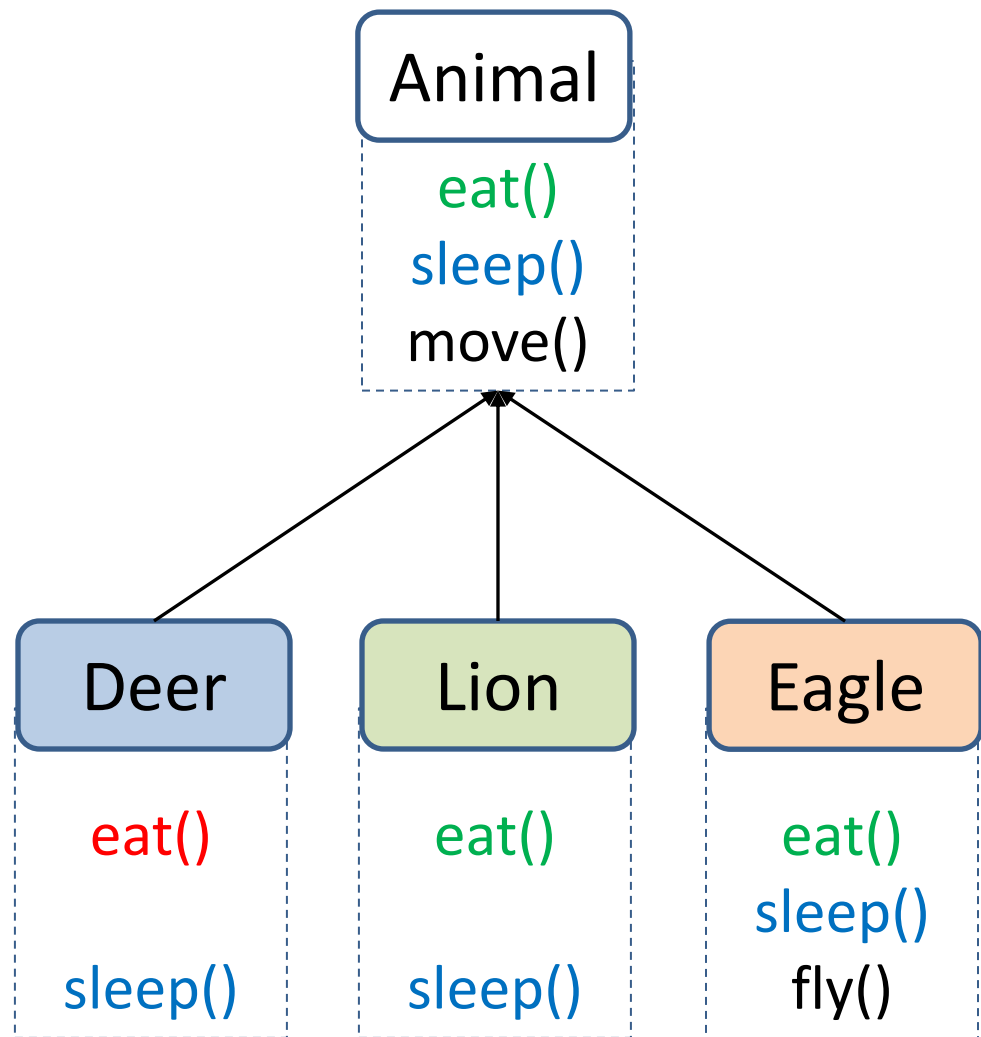
- 代码重复，代码臃肿；
- 代码可维护性差，未来有更多的动物加入这个大家庭怎么办？
- 违背了我们当初设计面向对象编程语言的初衷。
 - 可维护
 - 可扩展
 - 代码复用
- 所以，我们要对类进行下一步的优化，就是再抽象操作，或者说刚才的解决方案所做的抽象不够好。

用继承解决上述问题

我们能否再把类进行抽象一下，抽取一些相同的特质，组成一个动物类呢？

答案是：可以的，抽象提取出合适的父类，通过继承解决上述问题

这不是最好的解决方案，有缺陷



用继承解决上述问题

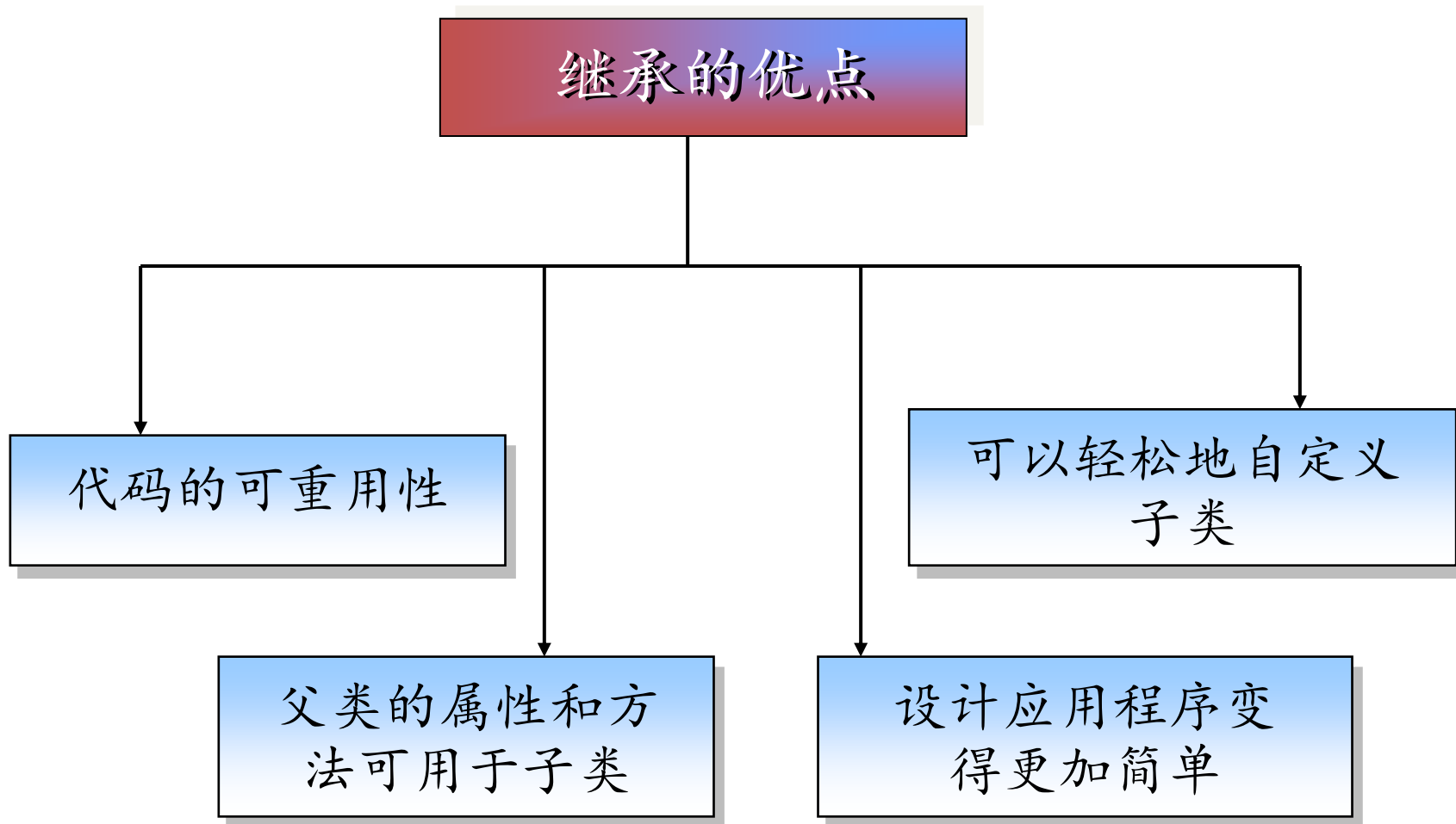
```
public class Animal{  
    public void eat(){  
        System.out.println(x: "我在吃肉");  
    }  
    public void sleep(){  
        System.out.println(x: "我在躺着睡觉");  
    }  
    public void move(){  
    }  
}
```

```
class Deer extends Animal {  
    public void eat(){  
        System.out.println(x: "我在吃肉");  
    }  
}
```

```
class Lion extends Animal {  
}  
  
class Eagle extends Animal {  
    public void fly(){  
        System.out.println(x: "I am flying!");  
    }  
}
```

- 减少代码重复
- 增强代码可维护性

为什么使用继承？



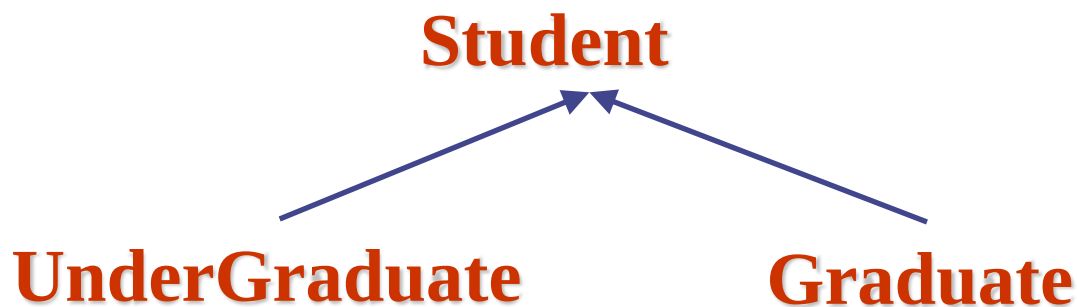
主要内容

- 继承的必要性
- 深入理解继承
 - 基类与派生类的关系
 - 单继承、多继承
 - 继承的层次结构
 - 子类继承了什么？
- Java继承（extends）
 - 子类构造方法
 - 保护成员的使用
 - 向上映射
 - 隐藏、覆盖
- 继承的优缺点、组合与继承
- 继承的案例

继承是一种“is-a”的关系

为什么本科生和研究生都有姓名、学号、地址，
而且都可以进行选课呢？

因为他们都是学生



这种 “是(is-a)” 的关系就是 继承

继承是一种“is-a”的关系

- 这种is-a关系，利用继承来实现
 - A is a B (或者A is kind of B)
 - A继承B
- 例如：
 - DateTime(日期时间类)既是Time(时间类)，也是Date(日期类)
 - 食肉动物(Carnivore类)是动物(Animal类)，食草动物(Herbivore类)也是动物(Animal类)

细说继承

- 继承是使用已存在的类的定义作为基础，建立新类的技术；
- 通过继承和派生形成的类族，反映了面向对象问题域、主题等概念；
- 继承是在保留原有类的数据成员和成员函数的基础上，派生出新类，新的类可以有某种程度的变异；
- 通过继承，新类自动具有了原有类的数据成员和成员函数，因而只需定义原有类型没有的新的数据成员和成员函数。实现了软件复用，使得类之间具备了层次性。

继承原有类的数据成员和成员函数

```
public class Animal{  
    String name;  
    public void eat(){  
        System.out.println(x: "我在吃肉");  
    }  
    public void sleep(){  
        System.out.println(x: "我在躺着睡觉");  
    }  
    public void move(){  
    }  
}
```

```
class Deer extends Animal {  
    public void eat(){  
        System.out.println(x: "我在吃草");  
    }  
    public String toString(){  
        return "My name is " + name;  
    }  
}
```

类的继承

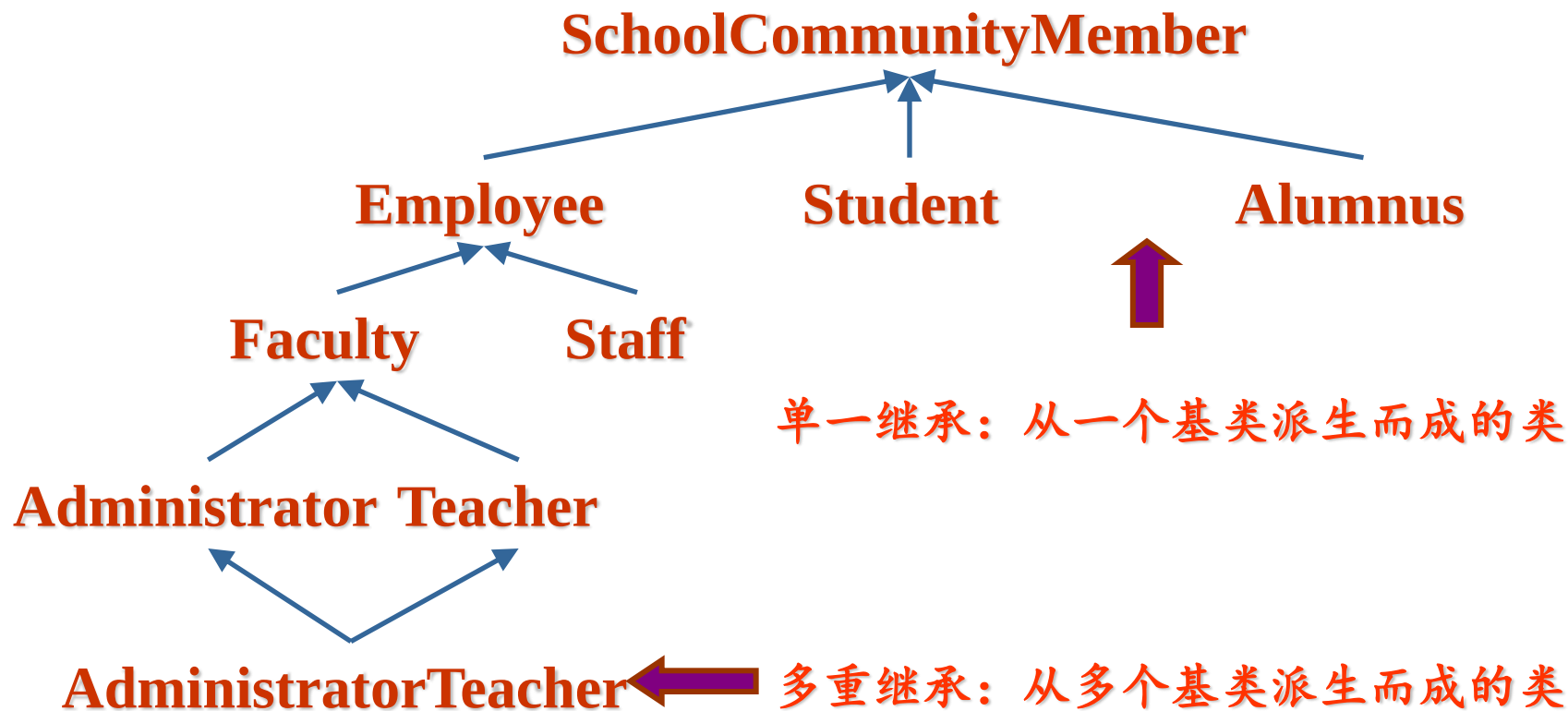
- 一. 被继承的类称为父类 (superclass), 继承后产生的类称为子类 (subclass)。
- 二. 单继承: 如果子类只能有一个直接父类, 称为单继承。
 - ◆ 例如, 轮船 --> 客轮; 人 --> 学生。
- 三. 多继承: 如果子类可以有多个直接父类, 称为多继承。
 - DateTime (日期时间类) 既是 Time (时间类), 也是 Date (日期类)
 - 沙发床是沙发和床的子类

基类和派生类的关系

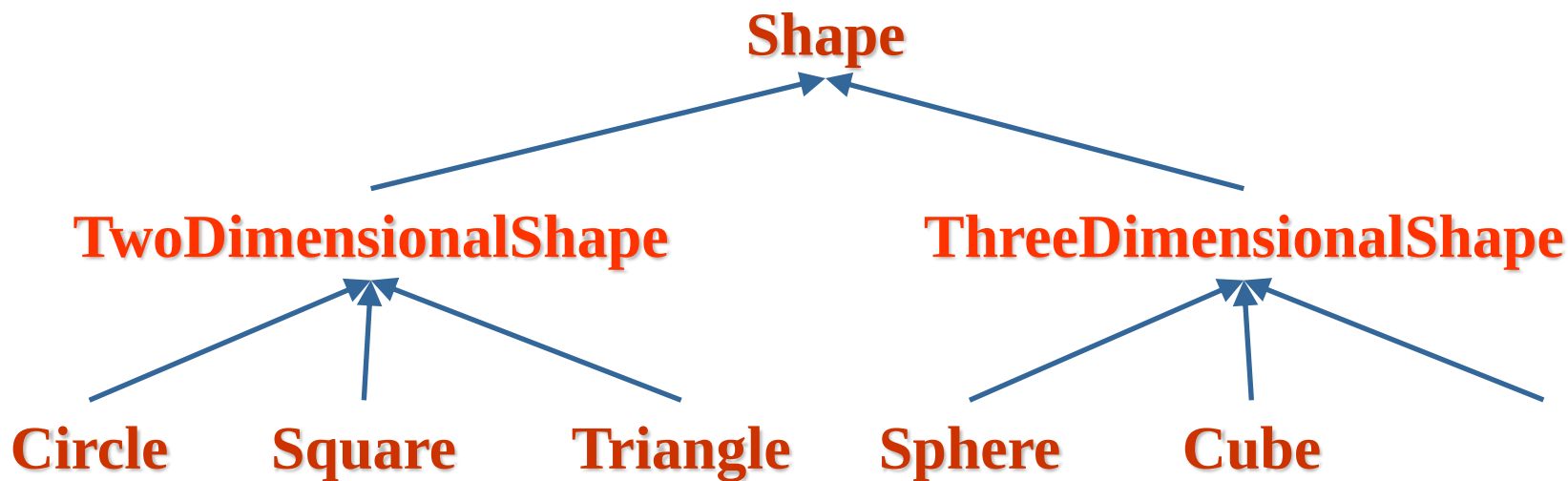
- 基类是对若干个派生类的抽象
 - 基类抽取了派生类的公共特征，在设计类时，**应该将通用的方法放到超类中**
 - Java中所有类共同的基类是**Object**类
- 派生类是基类的具体化
 - 通过扩展超类定义子类的时候，仅需指出子类与超类的不同之处，**通过增加数据成员或成员函数将基类变为某种更有用的类型**
- **派生类可以看作基类定义的延续**

继承层次结构-示例1

基类和派生类构成类的层次结构



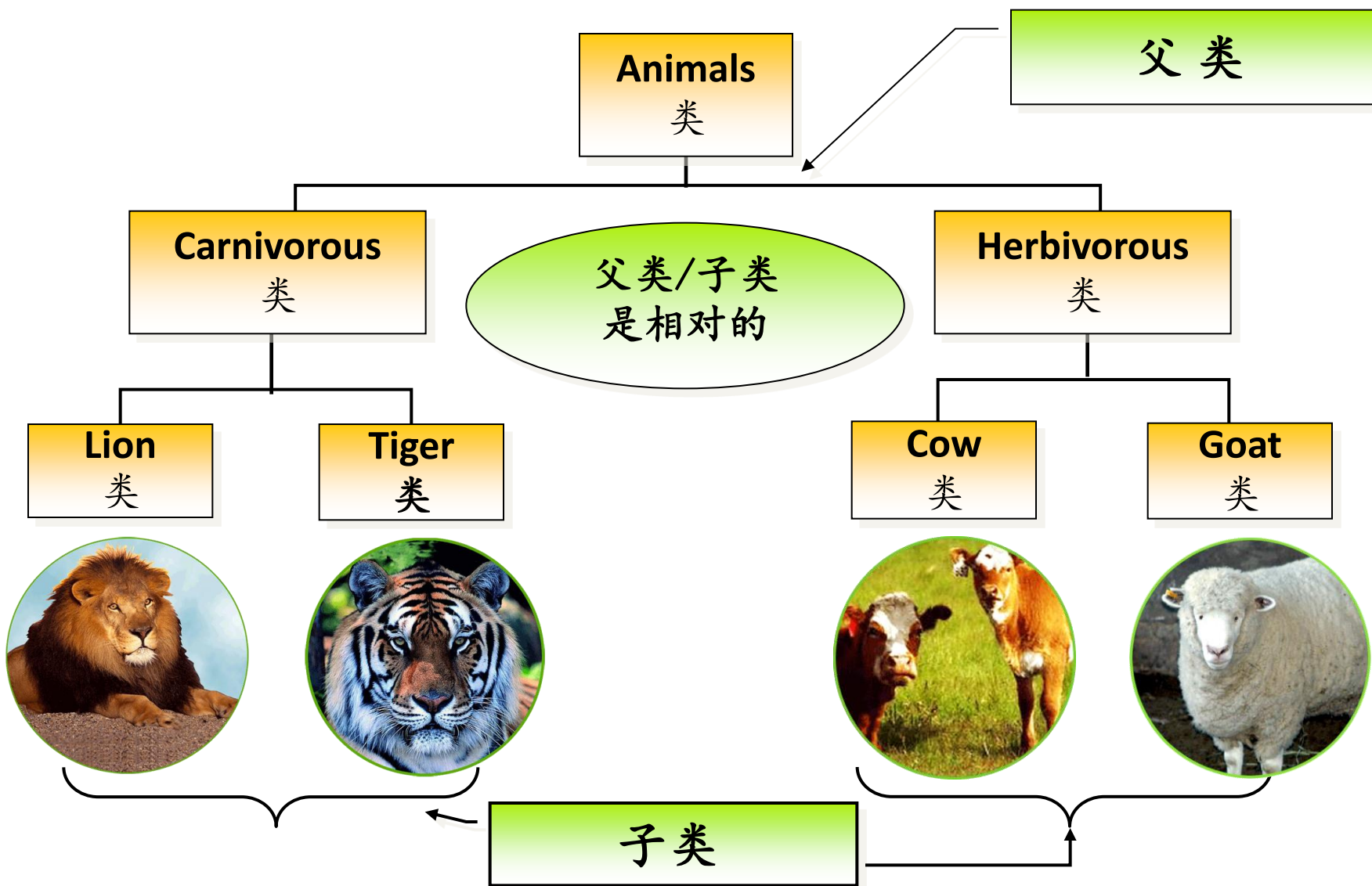
继承的层次结构-示例2



对现实世界中的层次结构进行分门别类！

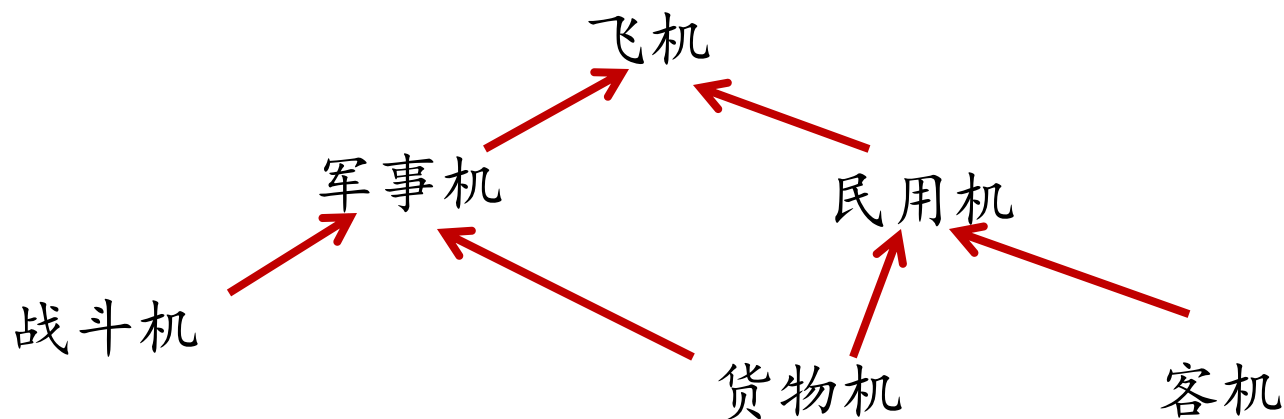
直接基类、直接派生类：Shape与TwoDimensionalShape
间接基类、间接派生类：Shape与Circle

继承的层次结构-示例3



问答题：请确认各类之间的关系

- Plane （飞机）
- military plane （军事机）
- passenger plane （客机）
- cargo plane （货物机）
- fighter plane （战斗机）
- Airliner （民用机）



子类从父类那里继承了什么？

- 一、子类拥有父类的所有属性和方法，只不过父类的私有属性和方法，子类是无法直接访问到的。
- 二、子类可以对父类进行扩展。子类可以拥有自己属性和方法；子类既可以重载父类的方法，也可以覆盖父类的方法。
- 子类不能继承父类的构造方法
— 如何解决？

继承的进一步理解

- 一. 继承避免了公用代码的重复开发，减少代码的冗余，提高程序的复用性；
- 二. 继承是类实现可重用性和可扩充性的关键特征。在继承关系下类之间组成网状的层次结构。
- 三. 通过继承增强一致性，从而减少模块间的接口和界面。
- 四. 通过向上映射(支持多态)，提高程序的可扩展性；

主要内容

- 继承的必要性
- 理解继承
 - 基类与派生类的关系
 - 单继承、多继承
 - 继承的层次结构
 - 子类继承了什么？
- Java继承 (extends)
 - 子类构造方法
 - 保护成员的使用
 - 向上映射
 - 隐藏、覆盖
- 继承的优缺点、组合与继承
- 继承的案例

- 一. Java不支持类的多继承。
- 二. Java中的继承通过关键字**extends**实现。
- 三. 类继承的格式：

```
[修饰符]class 类名 extends 父类名  
{  
    类体;  
}
```

- 一. **this**变量代表对象本身。
- 二. 当类中有两个同名变量，一个属于类的成员变量，而另一个属于方法中的局部变量，使用**this**区分成员变量和局部变量。
- 三. 使用**this**简化构造函数的调用。

This调用构造函数

1. 引用自身对象的成员变量
 - ◆ `this.age;`
2. 引用自身对象的成员方法
 - ◆ `this.diaplay();`
3. 调用自身的构造方法
 - ◆ `this("Jack",Male,10);`

```
public class Animal{
    String name;
    protected String type = "Animal";
    public Animal(){
        System.out.println(x: "Animal's Constructor 1 is running!");
    }
    public Animal(String name){
        this();
        this.name = name;
        System.out.println(x: "Animal's Constructor 2 is running!");
    }
}
```

super 关键字

- 一. 如果子类调用父类的构造函数，则通过 *super()* 调用来实现。
- 二. 如果子类调用父类的同名方法，则通过 *super. 方法名()* 来实现。

super 的用法

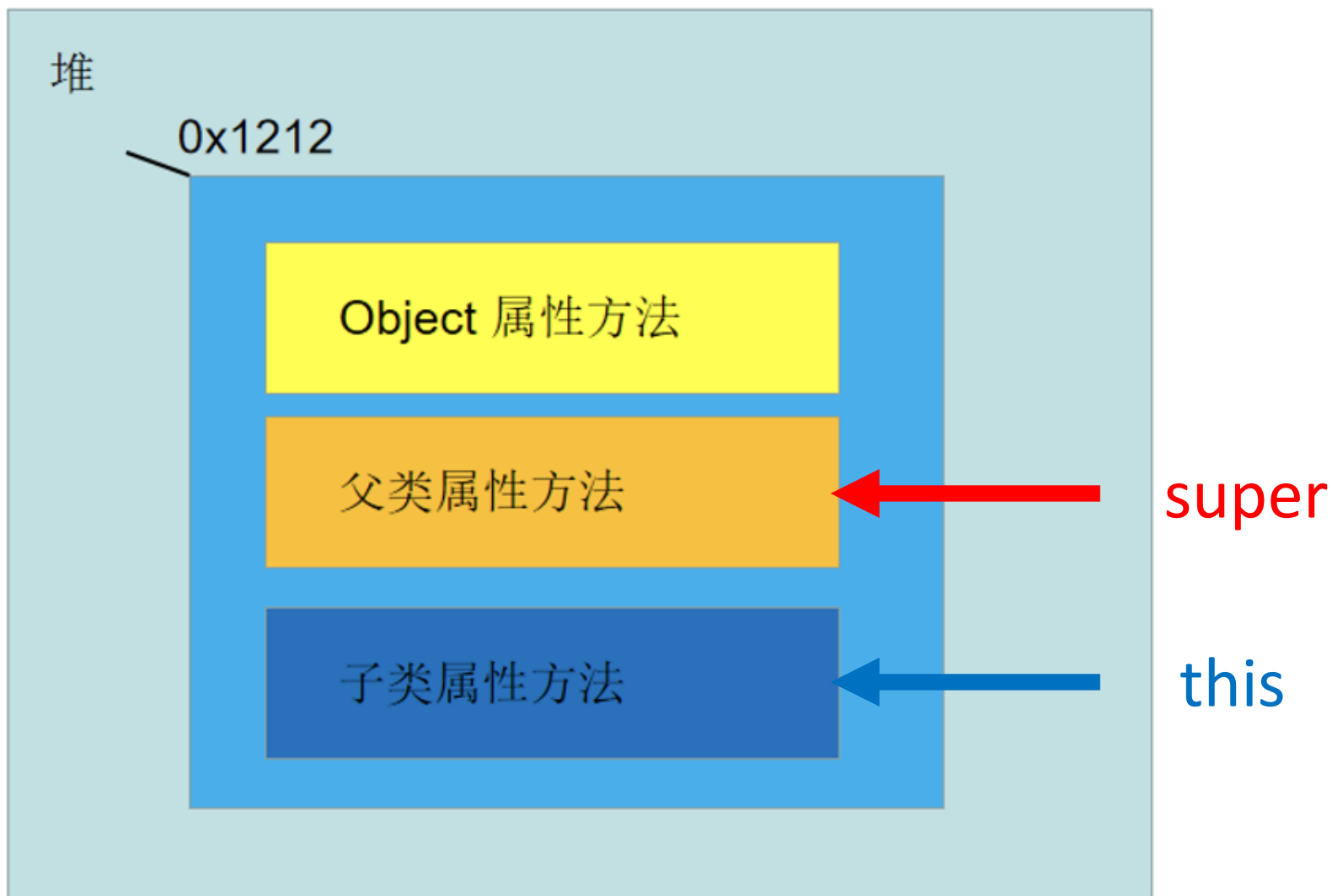
1. super用于:
2. 引用父类对象的成员变量
 - ◆ super.age;
3. 引用父类对象的成员方法
 - ◆ super.diaplay();
4. 调用父类的构造方法
 - ◆ super("Jack" ,Male,10);

```
public class Deer extends Animal {  
    public Deer(String name){  
        super(name);  
        System.out.println(x: "Deer's constructor 1 is running");  
    }  
}
```

this引用与super引用的对比

- Java 每个类都默认地具有 `null`、`this`、`super` 三个域，所以在任何类中都可以不加说明就可以直接引用它们：
 1. `null` : 代表“空”，用在定义一个对象但尚未为其开辟内存空间时。
 2. `this` 和 `super` : 是常用的指代子类对象和父类对象的关键字

this引用与super引用的对比



Java继承的三个重点

1. 编写子类构造器

- 子类不能继承父类的构造方法

2. protected关键字的使用

3. 向上转型

子类构造函数：

- 构造函数不能被继承；
- 无参子类构造函数的编写
 - 子类可以通过`super()`显示调用父类无参的构造函数，也可以隐式调用

无参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String name){
        this.name = name;
        System.out.println(x: "Constructor 2 is running!");
    }
}
```

Constructor 1 is
running!

```
public class Deer extends Animal {
    public Deer(){
        super();
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer();
    }
}
```

无参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String
        this.name = name;
        System.out.printl
    }
}
```

Constructor 1 is
running!

```
public class Deer extends Animal {
    public Deer(){
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}
class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer();
    }
}
```

无参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String name){
        this.name = name;
        System.out.println(x: "Constructor 2 is running!");
    }
}

public class Deer extends Animal {
    public Deer(String name){
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer(name: "little deer");
    }
}
```

Constructor 1 is
running!

无参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String name){
        this.name = name;
        System.out.println(x: "Constructor 2 is running!");
    }
}

public class Deer extends Animal {
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer();
    }
}
```

Constructor 1 is
running!

无参子类构造函数

```
public class Animal{
    String name;
public Animal(){
    System.out.println(x: "Constructor 1 is running!");
}
    public Animal(String name){
        this.name = name;
        System.out.println("Animal: " + name);
    }
}

public class Deer extends Animal {
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer();
    }
}
```

Compilation Error! ! !

子类构造函数：

- 构造函数不能被继承；
- 无参子类构造函数的编写
 - 子类可以通过`super()`显示调用父类无参的构造函数，也可以隐式调用
- 有参子类构造函数的编写
 - 初始化父类的成员变量；
 - 初始化子类的成员变量
 - 必须显示调用父类有参构造函数

有参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.pri
    }
    public Animal(Stri
        this.name = na
        System.out.pri
    }
```

Constructor 2 is
running!

```
public class Deer extends Animal {
    public Deer(String name){
        super(name);
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}

class Main{
    Run | Debug
    public static void main(String[] args){
        Deer deer = new Deer(name: "little deer");
    }
}
```


子类构造函数：

- 构造函数不能被继承；
- 无参子类构造函数的编写
 - 子类可以通过`super()`显示调用父类无参的构造函数，也可以隐式调用
- 有参子类构造函数的编写
 - 初始化父类的成员变量；
 - 初始化子类的成员变量
 - 必须显示调用父类有参构造函数
- 无论使用`this`调用本类构造函数，还是使用`super`调用父类构造函数，都必须是该方法体中的第一条可以执行语句，否则会产生语法错误。

有参子类构造函数

```
public class Animal{
    String name;
    public Animal(){
        System.out.println(x: "Constructor 1 is running!");
    }
    public Animal(String name){
        this.name = name;
        System.out.println(x: "Constructor 2 is running!");
    }
```

Compilation Error

```
public class Deer extends Animal {
    public Deer(String name){
        System.out.println(x: "Deer's constructor is running");
        super(name);
    }
    public void eat(){
        System.out.println(x: "我在吃草");
    }
    public String toString(){
        return "My name is " + name;
    }
}
```

子类对象的生成（构建）

- 子类创建对象时，**子类的构造方法**总是先调用父类的某个构造方法，完成父类部分的创建；然后再调用子类自己的构造方法，完成子类部分的创建。
- 如果子类的构造方法没有明显地指明使用父类的哪个构造方法，子类就调用父类的不带参数的构造方法。
- 子类在创建一个子类对象时，不仅子类中声明的成员变量被分配了内存，而且父类的所有的成员变量也都分配了内存空间。

子类对象的生成(构造函数的调用顺序)

- 创建子类对象时，子类总是按层次结构从**上到下**的顺序调用所有超类的构造函数。
- 如果父类没有不带参数的构造方法，则在子类的构造方法中必须明确的告诉调用父类的某个带参数的构造方法，通过**super**关键字，这条语句还必须出现在构造方法的第一句。

子类对象的创建-父类对象

```
public class Animal{  
    String name;  
    public Animal(){  
        System.out.println(x: "Constructor 1 is running!");  
    }  
    public Animal(String name){  
        this.name = name;  
        System.out.println(x: "Constructor 2 is running!");  
    }  
}
```

子类对象的创建

```
public class Deer extends Animal {  
    public Deer(String name){  
        super(name);  
        System.out.println(x: "Deer's constructor 1 is running");  
    }  
    public Deer(String name, int age){  
        this(name);  
        System.out.println(x: "Deer's constructor 2 is running");  
    }  
    public void eat(){  
        System.out.print  
    }  
    public String toString()  
        return "My name  
    }  
}  
  
class Main{  
    Run | Debug  
    public static void main(String[] args){  
        Deer deer = new Deer(name: "little deer");  
        System.out.println(deer);  
    }  
}
```

Animal's Constructor 2 is running!
Deer's constructor 1 is running
My name is little deer



子类对象的创建

```
public class Deer extends Animal {  
    public Deer(String name){  
        super(name);  
        System.out.println(x: "Deer's constructor 1 is running");  
    }  
    public Deer(String name, int age){  
        this(name);  
        System.out.println(x: "Deer's constructor 2 is running");  
    }  
    public void eat(){  
        System.out.print  
    }  
    public String toString()  
        return "My name  
    }  
}  
  
class Main{  
    Run | Debug  
    public static void main(String[] args){  
        Deer deer = new Deer(name: "little deer", age: 2);  
        System.out.println(deer);  
    }  
}
```

Animal's Constructor 2 is running!
Deer's constructor 1 is running
Deer's constructor 2 is running
My name is little deer

子类对象的创建

```
public class Deer extends Animal {  
    public Deer(String name){  
        super(name);  
        System.out.println(x: "Deer's constructor 1 is running");  
    }  
    public Deer(String name, int age){  
        System.out.println(x: "Deer's constructor 2 is running");  
    }  
    public void eat(){  
        System.out.println(x: "我在吃草");  
    }  
    public String toString(){  
        return "My name is little deer";  
    }  
}  
  
class Main{  
    Run | Debug  
    public static void main(String[] args){  
        Deer deer = new Deer(name: "little deer", age: 2);  
        System.out.println(deer);  
    }  
}
```

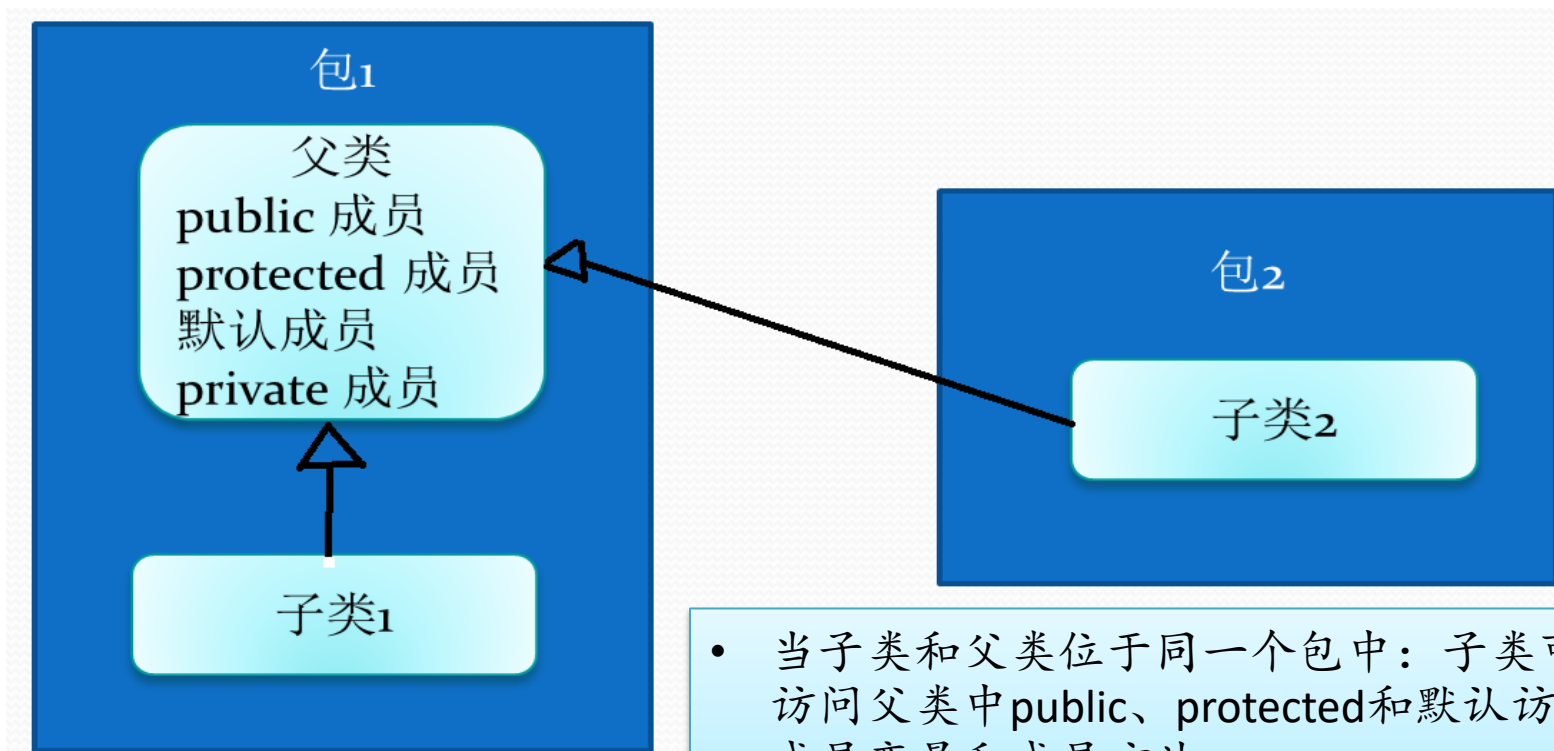
Animal's Constructor 1 is running!
Deer's constructor 2 is running
My name is little deer

保护成员的使用

继承中的访问权限

Modifier	Same Class	Same Package	Subclass	Universe
private	Yes			
default	Yes	Yes		
protected	Yes	Yes	Yes	
public	Yes	Yes	Yes	Yes

继承中的访问权限



- 当子类 and 父类位于同一个包中：子类可以直接访问父类中public、protected和默认访问级别的成员变量和成员方法。
- 当子类 and 父类位于不同的包中：情况比较复杂，具体问题，具体分析。

Java中的protected修饰符

1. 父类的protected成员是包内可见的，并且对子类可见；
2. 若子类与父类不在同一包中，那么在子类中，子类实例可以访问其从父类继承而来的protected方法，而不能访问父类实例的protected方法。

父类中protected成员函数

```
package cn.edu.buaa.oop.lesson_5;

public class Parent {
    private String privateField = "private field";
    protected String protectedField = "protected field";
    public String publicField = "public field";

    protected void getMessage(){
        System.out.println(x: "Parent's genMessage");
        System.out.println(protectedField);
    }
}
```

在子类中通过该子类引用可以访问其 protected 方法

```
package cn.edu.buaa.oop.lesson_5;

public class Son1 extends Parent{
    Run | Debug
    public static void main(String []args){
        Son1 son1 = new Son1();
        son1.message();
        son1.getMessage();
    }
    private void message(){
        super.getMessage();
        System.out.println(x: "Son1's message");
    }
}
```

不同包下，在子类中通过该子类引用可以访问其 **protected** 方法

```
package cn.edu.buaa.oop.lesson_5.subpackage;

import cn.edu.buaa.oop.lesson_5.Parent;

public class Son2 extends Parent{
    Run | Debug
    public static void main(String []args){
        Son2 son2 = new Son2();
        son2.message();
        son2.getMessage();
    }
    private void message(){
        super.getMessage();
        System.out.println(x: "Son1's message");
    }
}
```

子类中实际上把父类的方法继承下来了，可以通过该子类对象访问，也可以在子类方法中直接访问。还可以通过 **super** 关键字调用父类中的该方法。

不同包下，在子类中通过父类引用不可以访问其 protected 方法

```
package cn.edu.buaa.oop.lesson_5.subpackage;

import cn.edu.buaa.oop.lesson_5.Parent;

public class Son2 extends Parent{
    Run | Debug
    public static void main(String []args){
        Son2 son2 = new Son2();
        son2.message();
        son2.getMessage();
        Parent parent = new Parent();
        parent.getMessage();
    }
    private void message(){
        super.getMessage();
        System.out.println(x: "Son1's message");
    }
}
```


Java中的protected修饰符

1. 父类的protected成员是包内可见的，并且对子类可见；
2. 若子类与父类不在同一包中，那么在子类中，子类实例可以访问其从父类继承而来的protected方法，而不能访问父类实例的protected方法。
3. 不同包下，在子类中不能通过另一个子类引用访问共同基类的protected方法

```
package cn.edu.buaa.oop.lesson_5.subpackage1;
```

```
import cn.edu.buaa.oop.lesson_5.Parent;
```

```
public class Son2 extends Parent{
```

```
    package cn.edu.buaa.oop.lesson_5.subpackage2;
```

```
    import cn.edu.buaa.oop.lesson_5.Parent;
```

```
    import cn.edu.buaa.oop.lesson_5.subpackage1.Son2;
```

```
    public class Son3 extends Parent{
```

Run | Debug

```
        public static void main(String []args){
```

```
            Son2 son2 = new Son2();
```

```
            son2.getMessage();
```



```
        }
```

```
        private void message(){
```

```
            super.getMessage();
```

```
            System.out.println(x: "Son1's message");
```

```
        }
```

```
    }
```

Java中的protected修饰符

1. 父类的protected成员是包内可见的，并且对子类可见；
2. 若子类与父类不在同一包中，那么在子类中，子类实例可以访问其从父类继承而来的protected方法，而不能访问父类实例的protected方法。
3. 不同包下，在子类中不能通过另一个子类引用访问共同基类的protected方法
4. 对于protected修饰的静态变量，无论是否同一个包，在子类中均可直接访问

继承中的静态变量

```
package cn.edu.buaa.oop.lesson_5;
```

```
public class Parent {
```

```
    private String privateField = "private field";
```

```
    protected String protectedField = "protected field";
```

```
    public String publicField = "public field";
```

```
    protected static String staticField = "static field";
```

```
    protected static package cn.edu.buaa.oop.lesson_5;
```

```
        System.out.pr
```

```
        System.out.pr public class Kid1 extends Parent{
```

Run | Debug

```
    public static void main(String []args){
```

```
        System.out.println(Kid1.staticField);
```

```
        System.out.println(Parent.staticField);
```

```
        Kid1.staticField = "Kid1's static field";
```

```
        System.out.println(Kid1.staticField);
```

```
        System.out.println(Parent.staticField);
```

static field

static field

Kid1's static field

Kid1's static field

继承中的静态变量

```
package cn.edu.buaa.oop.lesson_5;
```

```
public class Parent {  
    private String privateField = "private field";  
    protected String protectedField = "protected field";  
    public String publicField = "public field";  
    protected static String staticField = "static field";  
}
```

```
protected static void  
    System.out.print  
    System.out.print  
}
```

```
package cn.edu.buaa.oop.lesson_5.subpackage1;
```

```
import cn.edu.buaa.oop.lesson_5.Parent;
```

```
public class Kid2 extends Parent{
```

Run | Debug

```
public static void main(String []args){  
    System.out.println(Kid2.staticField);  
    System.out.println(Parent.staticField);  
}
```

```
Kid2.staticField = "Kid2's static field";  
System.out.println(Kid2.staticField);  
System.out.println(Parent.staticField);
```

static field
static field
Kid2's static field
Kid2's static field

Java中的protected修饰符

1. 父类的protected成员是包内可见的，并且对子类可见；
2. 若子类与父类不在同一包中，那么在子类中，子类实例可以访问其从父类继承而来的protected方法，而不能访问父类实例的protected方法。
3. 不同包下，在子类中不能通过另一个子类引用访问共同基类的 protected 方法
4. 对于protected修饰的静态变量，无论是否同一个包，在子类中均可直接访问
5. 对于protected修饰的静态函数，无论是否同一个包，在子类中均可直接访问。在不同包的非子类中则不可访问。

父类中protected静态函数

```
package cn.edu.buaa.oop.lesson_5;
```

```
public class Parent {  
    private String privateField = "private field";  
    protected String protectedField = "protected field";  
    public String publicField = "public field";  
    protected static String staticField = "static field";
```

```
    protected static void getStaticMessage(){  
        System.out.println(x: "Parent's getStaticMessage");  
        System.out.println(staticField);  
    }
```

子类均可直接访问父类静态成员函数

```
package cn.edu.buaa.oop.lesson_5.subpackage1;
```

```
import cn.edu.buaa.oop.lesson_5.Parent;
```

```
public class Kid2 extends Parent{
```

Run | Debug

```
    public static void main(String []args){
```

```
        Parent parent = new Parent();
```

```
        parent.getMessage();
```

```
        Parent.getStaticMessage();
```



在不同包下，非子类不可访问

```
package cn.edu.buaa.oop.lesson_5.subpackage1;  
import cn.edu.buaa.oop.lesson_5.Parent;
```

```
public class Kid4 {
```

Run | Debug

```
    public static void main(String []args){
```

```
        Parent parent = new Parent();
```

```
        parent.getMessage();
```



```
        Parent.getStaticMessage();
```



```
    }
```

```
}
```

继承中public和默认访问级别

- Java中，类可以是**public和默认访问级别**
 - public级别的类可以被所有其他类继承。
 - 默认级别的类只能被同一个包中的类继承。

示例代码(默认级别的类只能被同一个包中的类访问)

```
package mpack1;  
class Base1{}  
public class Base2{}
```

```
package mpack2;
```

```
class Sub1 extends Base1{} //非法
```

```
class Sub2 extends Base2{}
```

```
public class Guest{  
    public void test(){  
        Base1 b1=new Base1(); //非法  
        Base2 b2=new Base2();  
    }  
}
```

Quiz

1. 同一个包中，子类对象能访问父类的 protected 方法吗？

◆ 在同一个包中，普通类或者子类都可以访问基类的 protected 方法。

2. 不同包下，在子类中创建该子类对象能访问父类的 protected 方法吗？

◆ 可以

3. 不同包下，在子类中创建父类对象能访问父类的 protected 方法吗？

◆ 不可以

4. 不同包下，在子类中创建另一个子类的对象能访问公共父类的 protected 方法吗？

◆ 不可以

5. 不同包下，子类能访问父类的 protected 静态方法吗？

◆ 可以

继承中的数据保护原则

- 将基类数据成员定义成private
 - 派生类中通过相应的访问函数进行访问
 - 优点：保持基类的封装性
 - 缺点：降低了访问效率

继承中的数据保护原则

- 保护成员会破坏基类的封装性
 - 派生类可以直接访问和修改
- 关于保护成员的使用
 - 如果基类仅向其派生类提供服务，而不对其他客户提供该服务，使用protected成员访问说明符是合适的
 - 例如Owner和Son、Wife、Daughter是父子关系，Owner的车和房产可以被子类直接访问，但是不对其他类开放，那么车和房产可以声明为protected的权限。

向上转型（重写与覆盖）

向上转型 (upcasting)

- 将子类转换成父类，在继承关系上面是向上移动的，所以一般称之为向上转型或者向上映射。
- 由于向上转型是从一个叫专用类型向较通用类型转换，所以它总是安全的

向上转型 (upcasting)

```
package cn.edu.buaa.oop.lesson_5;
```

```
public class Lion extends Animal {  
    public Lion(String name){  
        super(name);  
    }  
    public String toString(){  
        return "My name is " + super.name;  
    }  
}
```

```
class Main2{
```

Run | Debug

```
public static void main(String[] args){  
    Lion lion = new Lion(name: "little lion");  
    System.out.println(lion);
```

```
        Animal lion2 = lion;  
        System.out.println(lion2);
```

← 向上转型

向上转型的缺憾：

- 只能调用父类中定义的属性和方法，对于子类中的方法和属性它就望尘莫及了，必须**强制转成子类类型**
- 这就是为什么编译器在“未曾明确表示转型”或者“未曾指定特殊标记”的情况下，仍然允许向上转型的原因。

向上转型的缺憾:

```
public class Lion extends Animal {  
    public Lion(String name){  
        super(name);  
    }  
    public String toString(){  
        return "My name is " + super.name;  
    }  
    public void printName(){  
        System.out.println("My name is " + super.name);  
    }  
}
```

```
class Main2{
```

Run | Debug

```
public static void main(String[] args){  
    Lion lion = new Lion(name: "little lion");  
    lion.printName();  
  
    Animal lion2 = lion;  
    lion2.printName();  
}
```

printName is not defined
in parent class



- 一. 变量隐藏：在子类对父类的继承中，如果子类的成员变量和父类的成员变量同名，此时称为子类隐藏（`override`）了父类的成员变量。
- 二. 子类若要引用父类的同名变量。要用 `super` 关键字做前缀加圆点操作符引用，即 `super. 变量名`

变量隐藏

```
public class Animal{  
    String name;  
    protected String type = "Animal";  
}
```

```
public class Lion extends Animal {  
    private String type = "Lion";  
  
    public String getType(){  
        return type;  
    }  
  
    public String getSuperType(){  
        return super.type;  
    }  
}
```

变量隐藏

```
public static void main(String[] args){  
    Lion lion = new Lion(name: "little lion");  
    lion.printName();  
    System.out.println(lion.getType());  
    System.out.println(lion.getSuperType());  
  
    ... .. Animal lion2 = lion;  
    ... .. System.out.println(lion2.type);  
}
```

Lion
Animal
Animal

方法隐藏（重写）和方法覆盖

- 覆盖就是子类的方法跟父类的方法具有完全一样的签名（它们的名称、参数以及返回类型完全相同）
- 私有方法、静态方法不能被覆盖，如果在子类出现了同签名的方法，就是方法隐藏
- 用final声明的成员方法是最终方法，最终方法不能被子类覆盖（重写）

方法隐藏（重写）和方法覆盖

- 覆盖：调用的总是实例化对象的同名方法；
- 隐藏：调用的总是引用的类型的同名方法或同名变量；


```
package com.buaa.test;
```

The hide method in Planet.
The override method in Earth.

```
class Planet {
```

```
    public static void hide() {  
        System.out.println("The hide method in Planet.");  
    }
```

```
    public void override() {  
        System.out.println("The overrid method in Planet.");  
    }
```

```
}
```

```
public class Earth extends Planet {
```

```
    public static void hide() {  
        System.out.println("The hide method in Earth.");  
    }
```

```
    public void override() {  
        System.out.println("The override method in Earth.");  
    }
```

```
    public static void main(String[] args) {
```

```
        Earth myEarth = new Earth();
```

```
        Planet myPlanet = myEarth;
```

```
        myPlanet.hide();
```

```
        myPlanet.override();
```

```
    }
```

```
}
```

```
package com.buaa.test;
```

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        Subclass s = new Subclass();
```

```
        System.out.println(s.x + " " + s.y + " " + s.z);
```

```
        System.out.println(s.method());
```

```
        Base b2 = s; //正确
```

```
        Base b3 = (Base)s; //正确
```

```
        System.out.println(b2.x+" "+b2.y+" "+b2.z);
```

```
        System.out.println(b2.method());
```

```
    }
```

```
}
```

```
class Base {
```

```
    int x = 1;
```

```
    static int y = 2;
```

```
    int z = 3;
```

```
    int method() {
```

```
        return x;
```

```
    }
```

```
}
```

```
class Subclass extends Base {
```

```
    int x = 4;
```

```
    int y = 5;
```

```
    static int z = 6;
```

```
    int method() {
```

```
        return x;
```

```
    }
```

```
}
```



4 5 6

4

1 2 3

4

- 私有方法、静态方法不能被覆盖，如果在子类出现了同签名的方法，就是方法隐藏
- 用final声明的成员方法是最终方法，最终方法不能被子类覆盖（重写）

Why?

主要内容

- 继承的必要性
- 理解继承
 - 基类与派生类的关系
 - 单继承、多继承
 - 继承的层次结构
 - 子类继承了什么？
- Java继承（extends）
 - 子类构造方法
 - 保护成员的使用
 - 向上映射
 - 隐藏、覆盖
- 继承的优缺点、组合与继承
- 继承的案例

继承的好处与坏处

- 1、继承带来了哪些好处？
2、继承是否破坏了类的封装性？

面向对象的三大特征之一：**继承真的很好！**

- 一. 继承避免了公用代码的重复开发，减少代码的冗余，提高程序的复用性；
- 二. 支持向上映射，提高程序的可扩展性；
- 三. 继承是类实现可重用性和可扩充性的关键特征。在继承关系下类之间组成网状的层次结构。
- 四. 通过继承增强一致性，从而减少模块间的接口和界面。
- 五. 通过向上映射，让我们体验到多态的益处。**

继承是否破坏了类的封装性？

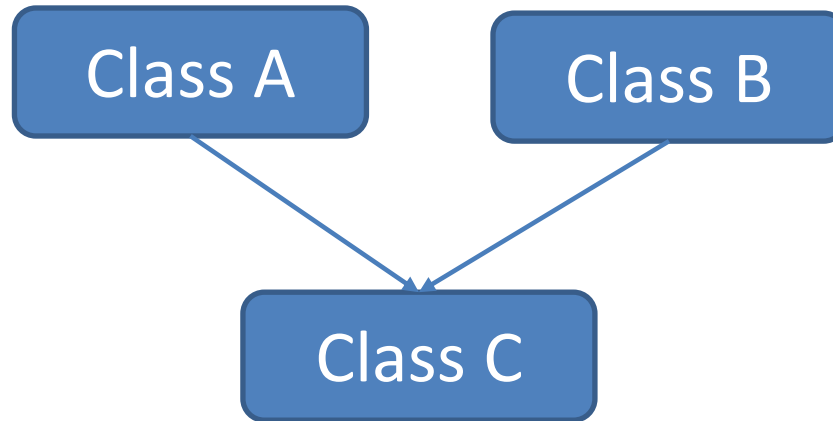
- 是的。继承破坏了封装性，换句话说，子类依赖于父类的实现细节。**继承很容易改变父类实现的细节(所以父类中能写成final尽量写成final)**，即使父类整体没有问题，也有可能因为子类细节实现不当，而破坏父类的约束。
- 其实这是一个平衡关系，不是绝对关系，一定程度的封装和一定程度的继承，可以提高开发效率，继承破坏了封装，但是有时继承是必须的，为了继承牺牲一定的封装是允许的。不能绝对的为了封装，就不去继承。

- 继承是一种强耦合关系。父类变，子类就必须变。
- 继承破坏了封装，子类可以重写父类的方法，子类可以直接访问保护成员。
- 那么到底要不要使用继承呢？
 - 《Think in java》中提供了解决办法：问一问自己是否需要从子类向父类**进行向上转型**。如果必须向上转型，则继承是必要的，但是如果不需要，则应当好好考虑自己是否需要继承。

继承的使用原则

- 合理使用继承，谨慎继承，**继承树的层次不可太多**
- 继承是一种提高程序代码的可重用性、以及提高系统的可扩展性的有效手段
- 但是，如果继承树非常复杂、或者随便扩展本来不是专门为继承而设计的类，反而会削弱系统的可扩展性和可维护性

多继承的钻石问题



```
class ClassA {  
    public void doSomething(){  
        System.out.println(x: "doSomething of A");  
    }  
}
```

```
class ClassB {  
    public void doSomething(){  
        System.out.println(x: "doSomething of B");  
    }  
}
```

多继承的钻石问题

```
class ClassA {  
    public void doSomething(){  
        System.out.println(x: "doSomething of A");  
    }  
}
```

```
class ClassB {  
    public void doSomething(){  
        System.out.println(x: "doSomething of B");  
    }  
}
```

```
class ClassC extends ClassA, ClassB {  
    public void test(){  
        //调用父类的方法  
        doSomething();  
    }  
}
```

歧义

组合 (has a)

```
class ClassD{
```

```
    ClassA objA = new ClassA();
```

```
    ClassB objB = new ClassB();
```

```
    public void test(){  
        objA.doSomething();  
    }
```

```
    public void methodA(){  
        objA.methodA();  
    }
```

```
    public void methodB(){  
        objB.methodB();  
    }
```



```
}
```

- 组合能解决钻石问题
- 组合不会破坏类的封装
- 组合能精确控制父类子类变量方法的访问权限
- 组合会创建更多的对象，占用内存

小结

- 基类与派生类的关系？ is a
- 如何继承一个类？ Extends
- 子类构造方法及子类对象的构建（super, this）
- 继承中保护成员的使用（protected）
- 向上映射
- 变量、方法的隐藏与覆盖
- 继承有什么好处、缺点？
- 什么是单继承？ 什么是多继承？ Java关于继承是如何定义的？